Pinacoteca
Documentazione di Progetto

Elettro5.B

March 15, 2026

Abstract

Questa documentazione descrive in modo completo il progetto Pinacoteca, con focus su architettura software, logica di controllo, simulazione e validazione. Sono inclusi requisiti, tracciabilita, motivazioni delle scelte, procedure di test e limiti noti, per garantire un quadro professionale e verificabile.

Contents

Abstract **i**

1 Introduzione **1**

2 Architettura **2**

2.1 Schema logico 2

2.2 Flusso dati 2

2.3 Cicli temporali 2

2.4 Parametri principali 2

3 Decisioni progettuali e sicurezza **3**

3.1 Modularita e responsabilita 3

3.2 Gestione errori sensori 3

3.3 Protezione attuatori e stabilita 3

3.4 Scelte di interfaccia e comunicazione 3

3.5 Strategia di test 3

4 Diagrammi **5**

4.1 Vista a blocchi 5

4.2 Sequenza logica del loop 5

5 Hardware **6**

5.1 Componenti 6

5.2 Criteri di scelta 6

5.3 Mappa pin 6

5.4 Schema grafico del cablaggio 7

5.5 Note di cablaggio 7

6 Firmware **8**

6.1 Setup 8

6.2 Loop 8

6.3 Gestione errori 8

6.4 Configurazione dei target 8

7 Moduli software **10**

7.1 LED 10
7.2 Servomotore e tornello 10
7.3 Semaforo 10
7.4 Temperatura e termostato 11
7.5 Umidita e controllo 11
7.6 Illuminazione 11

8 Display LCD **13**
8.1 Pagine 13
8.2 Calcolo data e ora 13
8.3 Aggiornamento 13
8.4 Sicurezza e robustezza 13

9 Implementazione passo-passo **14**
9.1 Inizializzazione globale 14
9.2 Tornello e conteggio persone 14
9.3 Semaforo di capienza 14
9.4 Termostato 15
9.5 Controllo umidita 15
9.6 Illuminazione 15
9.7 Display LCD 16

10 Requisiti e tracciabilita **17**
10.1 Requisiti funzionali 17
10.2 Tracciabilita test 17
10.3 Note 17

11 Test **18**
11.1 Struttura 18
11.2 Esecuzione 18
11.3 Motivazioni per i test 18
11.3.1 LED 18
11.3.2 Stoplight 18
11.3.3 Servomotore 19
11.3.4 Turnstile 19
11.3.5 Temperature 19

11.3.6 Thermostat . . . 19
11.3.7 Humidity e HumidifierControl . . . 19
11.3.8 Photoresistor e LightingControl . . . 20
11.3.9 DisplayInterface . . . 20
11.3.10 DisplayPanel . . . 20
11.4 Script run_tests.sh . . . 20

12 Istruzioni d'uso **21**
12.1 Build dei test . . . 21
12.2 Compilazione Arduino . . . 21
12.3 Simulazione Wokwi . . . 21

13 Rischi e failure modes **22**
13.1 Rischi sensori . . . 22
13.2 Rischi attuatori . . . 22
13.3 Rischi di integrazione . . . 22

14 Timing e prestazioni **23**
14.1 Frequenze di aggiornamento . . . 23
14.2 Motivazioni . . . 23
14.3 Impatto sul loop . . . 23

15 Limiti noti e roadmap **24**
15.1 Limiti attuali . . . 24
15.2 Roadmap . . . 24

16 Tracciabilita requisiti-design-test **25**
16.1 Matrice di tracciabilita . . . 25
16.2 Note . . . 25

17 Validazione software e simulazione **26**
17.1 Strategia di validazione . . . 26
17.2 Procedure di validazione in simulazione . . . 26
17.3 Criteri di accettazione software . . . 26

18 Calibrazione e limiti di simulazione **27**
18.1 Calibrazione software . . . 27
18.2 Limiti della simulazione . . . 27

19 Trade-off e scelte progettuali **28**
19.1 Gestione del tempo . . . 28
19.2 Valori sentinella per errori . . . 28
19.3 Architettura modulare . . . 28

20 Glossario **29**

21 Appendice: API pubbliche **30**
21.1 Turnstile . . . 30
21.2 Stoplight . . . 30
21.3 Thermostat . . . 30
21.4 HumidifierControl . . . 30
21.5 LightingControl . . . 31
21.6 DisplayPanel . . . 31
21.7 Funzioni di utilita . . . 31

22 Software e dipendenze **32**
22.1 Strumenti . . . 32
22.2 Librerie . . . 32
22.3 Repository . . . 32

23 Controllo remoto, comunicazione App e miglioramenti di stabilita **33**
23.1 Obiettivo del capitolo . . . 33
23.2 Architettura della comunicazione remota . . . 33
23.3 Protocollo utilizzato dall'app . . . 33
23.3.1 Comandi base . . . 33
23.3.2 Comandi setpoint . . . 33
23.3.3 Comandi override manuale . . . 34
23.3.4 Formato payload stato . . . 34
23.3.5 Gestione errori . . . 34
23.4 Strategia di validazione e test . . . 34
23.5 Ottimizzazioni software per stabilita . . . 34
23.6 Problemi riscontrati in Wokwi e soluzioni . . . 35
23.6.1 Problema A: mismatch board/firmware . . . 35
23.6.2 Problema B: chiave duplicata in `wokwi.toml` . . . 35
23.6.3 Problema C: servo non visibile in movimento . . . 35

23.7 Workflow build completo (comando unico) 35
23.8 Test app senza hardware 35
23.9 Nota finale di deploy 36

Capitolo 1— Introduzione

Il progetto **Pinacoteca** realizza un sistema di gestione per una galleria con controllo di accessi, clima, umidita, illuminazione e visualizzazione su LCD. L'obiettivo e integrare sensori e attuatori in un firmware Arduino modulare, con test unitari eseguibili su PC e un flusso di build ripetibile.

La documentazione e pensata per descrivere sia la soluzione tecnica sia le motivazioni delle scelte progettuali. Ogni sezione approfondisce il comportamento passo passo, evidenziando i punti di sicurezza, i compromessi adottati e le ragioni dietro alle API esposte.

Le funzionalita principali sono:

- conteggio ingressi e uscite con tornello servoassistito;
- segnalazione disponibilita posti con semaforo a LED;
- regolazione temperatura con modalita riscaldamento o raffreddamento;
- controllo umidita con soglia e attuatore (LED in simulazione);
- controllo luce ambientale con fotoresistenza e dimmer;
- pannello LCD 16x2 con pagine informative.

Il codice e strutturato in moduli indipendenti dentro la cartella `lib/`. Questa scelta riduce le dipendenze incrociate, rende il comportamento piu leggibile e semplifica la validazione attraverso test unitari.

Il sistema e costruito con un criterio di affidabilita: se una lettura sensore e invalida, i moduli di controllo adottano un comportamento di fallback che spegne le uscite, evitando stati pericolosi o non deterministici.

Capitolo 2— Architettura

Il firmware segue un ciclo di esecuzione deterministico: inizializzazione in `setup()` e aggiornamento periodico dei moduli in `loop()`. Ogni modulo espone i metodi `begin()` e `update()` per separare configurazione e logica runtime, riducendo il rischio di stati inconsistenti.

2.1 Schema logico

- **Turnstile**: legge i pulsanti IN/OUT, aggiorna il conteggio persone e muove il servo.
- **Stoplight**: visualizza su LED verde/rosso la disponibilita di posti.
- **Thermostat**: legge il sensore NTC e decide riscaldamento/raffreddamento.
- **LightingControl**: legge la fotoresistenza e regola il dimmer.
- **HumidifierControl**: legge il DHT22 e accende o spegne il controllo umidita.
- **DisplayPanel**: alterna pagine LCD con stato, data/ora e diagnostica.

2.2 Flusso dati

Il flusso segue tre passaggi costanti:

1. **Acquisizione**: letture analogiche o digitali dai sensori.
2. **Validazione**: controllo di errori e filtri sulle soglie.
3. **Attuazione**: LED, PWM o servo in base alle decisioni.

Il display aggrega gli stati dei moduli e presenta una vista sintetica per l'operatore.

2.3 Cicli temporali

Alcuni moduli usano tempi interni per evitare transizioni rapide o flicker. In particolare:

- il termostato applica una pausa prima di cambiare modalita;
- il display limita il refresh a 1 secondo e cambia pagina ogni 5 secondi.

Queste scelte evitano oscillazioni e migliorano la stabilita percepita.

2.4 Parametri principali

I valori di target sono configurati nel file principale:
`20.0 C` per temperatura, `65%` per umidita, `200 lux` per illuminazione, `5 persone` come capienza massima.

Capitolo 3— Decisioni progettuali e sicurezza

Questo capitolo spiega le scelte tecniche e le misure di sicurezza implementate. L'obiettivo e ottenere un firmware robusto, prevedibile e facilmente testabile.

3.1 Modularita e responsabilita

La scelta di suddividere il codice in moduli dentro `lib/` nasce da tre esigenze: isolare la logica dei sensori, semplificare il riuso e abilitare test unitari mirati. Ogni modulo ha un compito unico (single responsibility) e una superficie API ridotta.

La separazione tra `begin()` e `update()` permette di distinguere la configurazione hardware dal comportamento runtime. Questo riduce gli effetti collaterali e rende la logica osservabile anche nei test con mock.

3.2 Gestione errori sensori

Per sensori analogici e digitali si usano valori sentinella coerenti: `-999.0` per errori di lettura. Questa scelta e intenzionale per rendere immediata l'identificazione di un errore a valle, senza eccezioni o stati complessi.

Ogni modulo di controllo verifica le letture prima di attuare. Se l'input non e valido, il modulo spegne le uscite (fail-safe) e ritorna `false`. In questo modo un errore non si propaga in modo silenzioso.

3.3 Protezione attuatori e stabilita

Nel modulo servo, `antiSufferingServo` impedisce movimenti oltre i limiti meccanici, con un messaggio di log su Serial. Questa protezione evita blocchi o stress del motorino.

Il termostato introduce una pausa, definita da `_pauseTime`, per evitare il toggling rapido quando la lettura oscilla vicino alla soglia. Questo migliora affidabilita e durata delle uscite.

Il controllo luci usa un PWM limitato a 0–255. In simulazione Wokwi il valore e forzato a 255 quando serve luce, garantendo coerenza visiva e riducendo ambiguita nei test.

3.4 Scelte di interfaccia e comunicazione

Il display LCD e aggiornato con frequenza massima di 1 secondo e ruota le pagine ogni 5 secondi. Questa politica protegge dalla scrittura eccessiva, evita flicker e mantiene leggibilita.

La pagina errori sintetizza problemi operativi (umidita alta/bassa, temperatura anomala, conteggio persone fuori range) con testi brevi. La scelta e orientata alla diagnosi rapida in ambienti reali.

3.5 Strategia di test

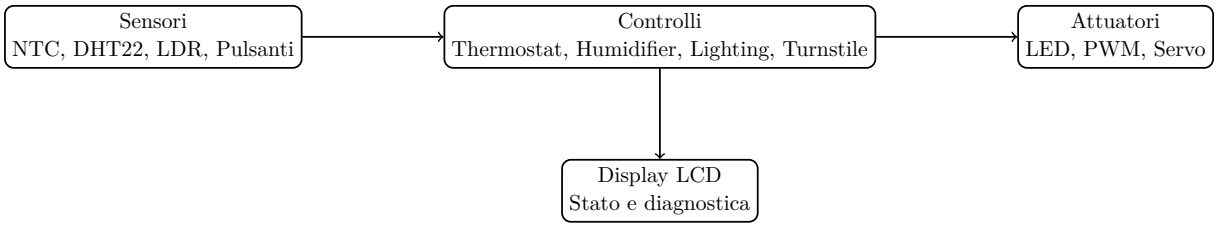
La suite di test e pensata per validare le condizioni di bordo: limiti di sensori, limiti attuatori, transizioni di stato e fallback di sicurezza. Ogni test verifica sia l'output logico sia il side effect sugli I/O mockati.

L'uso di mock locali Arduino permette di eseguire i test su PC, velocizzando il ciclo di feedback e riducendo la dipendenza dall'hardware. Questa scelta rende il progetto piu mantenibile e adatto a evoluzioni future.

Capitolo 4— Diagrammi

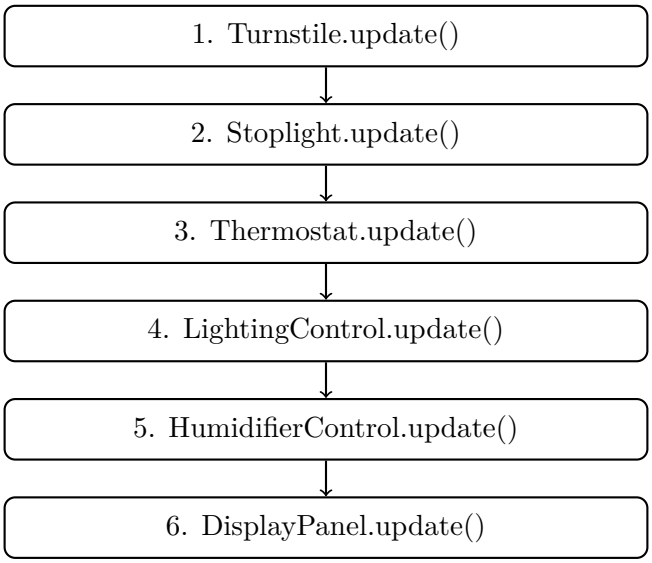
4.1 Vista a blocchi

La figura seguente riassume i blocchi principali e le interazioni tra sensori, controlli e attuatori.



4.2 Sequenza logica del loop

Il ciclo di aggiornamento e pensato per garantire che il display mostri sempre l'ultimo stato calcolato.



Capitolo 5— Hardware

La simulazione Wokwi descritta in `diagram.json` usa una Arduino Uno, un servo, pulsanti, LED, sensori e un LCD 16x2. Il cablaggio e coerente con i pin definiti nel firmware, cosi che la simulazione sia una rappresentazione fedele del comportamento atteso su hardware reale.

5.1 Componenti

- Arduino Uno
- Servo per tornello
- 2 pulsanti (IN/OUT) con resistori di pull-down
- LED verde e rosso (semaforo)
- LED blu e arancione (raffreddamento/riscaldamento)
- LED bianco (umidita)
- Fotoresistenza (LDR)
- Sensore NTC per temperatura
- Sensore DHT22 per umidita
- 4 LED gialli per plafoniera (pilotati in parallelo)
- LCD 16x2

5.2 Criteri di scelta

I componenti sono stati selezionati per semplicita di integrazione e disponibilita di librerie stabili. L'NTC offre una misura analogica lineare per il termostato, mentre il DHT22 fornisce umidita con interfaccia digitale semplice. L'LCD 16x2 rappresenta un compromesso tra costo e leggibilita, con un'interfaccia standardizzata tramite `LiquidCrystal`.

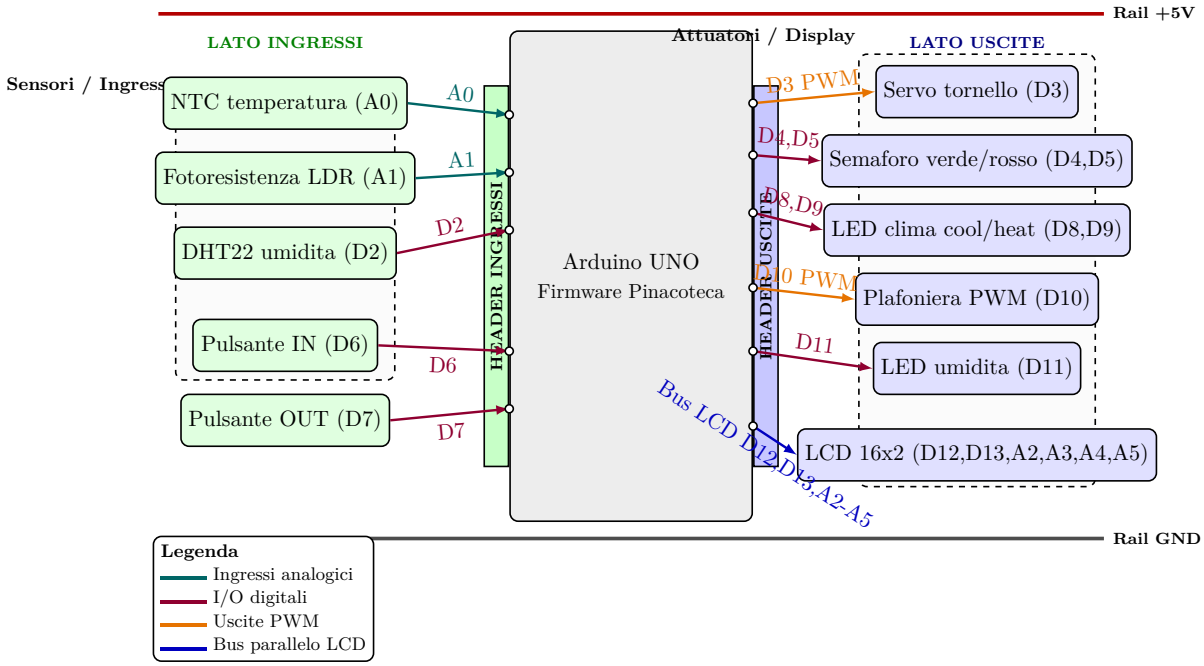
5.3 Mappa pin

Funzione	Pin	Note
Servo tornello	D3	PWM
Pulsante IN	D6	ingresso digitale
Pulsante OUT	D7	ingresso digitale
LED verde semaforo	D4	uscita digitale
LED rosso semaforo	D5	uscita digitale
LED raffreddamento	D8	uscita digitale
LED riscaldamento	D9	uscita digitale
Plafoniera (PWM)	D10	dimmer luci
LED umidita	D11	uscita digitale
Sensore temperatura (NTC)	A0	ingresso analogico
Fotoresistenza	A1	ingresso analogico
LCD RS	D12	
LCD EN	D13	
LCD D4	A2	

LCD D5	A3	
LCD D6	A4	
LCD D7	A5	
DHT22	D2	pin definito in `humidity.h`

5.4 Schema grafico del cablaggio

La figura seguente mostra una vista sintetica del cablaggio con Arduino al centro, sensori a sinistra e attuatori/display a destra. I collegamenti sono raggruppati per tipo di segnale (analogico, digitale, PWM, bus LCD) per migliorare la leggibilita mantenendo invariata la mappa pin firmware.



5.5 Note di cablaggio

- I LED sono collegati con resistori da 220 Ωper limitare la corrente.
- I pulsanti usano resistori da 10 kΩper pull-down, evitando letture flottanti.
- Il DHT22 usa il pin digitale 2 e richiede alimentazione a 5V.
- La plafoniera e pilotata in PWM sul pin D10, soluzione che semplifica la regolazione dell'intensita.

Capitolo 6— Firmware

Il file principale `Pinacoteca.ino` istanzia i moduli e definisce i pin. La logica e organizzata in due fasi ben distinte, in modo da avere un ciclo di esecuzione prevedibile e facilmente verificabile.

6.1 Setup

Durante `setup()` vengono inizializzati:

- seriale per debug e diagnostica;
- servo e tornello con posizione iniziale nota;
- semaforo, termostato, illuminazione e umidita;
- pannello LCD con messaggio di avvio.

La scelta di inizializzare prima il servo e poi gli altri moduli riduce il rischio di movimenti non voluti durante la configurazione dei pin.

6.2 Loop

Nel ciclo principale, ogni modulo aggiorna il proprio stato in una sequenza coerente:

1. aggiornamento tornello e conteggio persone;
2. aggiornamento semaforo in base al limite;
3. regolazione clima (riscaldamento o raffreddamento);
4. regolazione illuminazione;
5. controllo umidita;
6. refresh display con dati correnti.

L'ordine e intenzionale: si acquisiscono prima gli input umani, poi si aggiornano gli indicatori di stato e infine si aggiorna l'interfaccia LCD. In questo modo il display riflette l'ultima decisione del ciclo.

6.3 Gestione errori

I moduli che leggono sensori controllano la validita della misura. Se l'input non e valido, le uscite sono poste in stato sicuro e la funzione di aggiornamento ritorna `false`. Questo comportamento e coerente con la filosofia fail-safe del progetto.

6.4 Configurazione dei target

I target sono passati ai costruttori dei moduli:

- `Thermostat`: temperatura desiderata in gradi Celsius.
- `LightingControl`: illuminamento target in lux.
- `HumidifierControl`: umidita target in percento.
- `Turnstile`: numero massimo di persone.

La configurazione centralizzata nel file principale rende immediata la taratura dell'impianto senza modificare i singoli moduli.

Capitolo 7— Moduli software

Questa sezione descrive in modo dettagliato i moduli presenti in `lib/`. Ogni modulo e progettato con un'interfaccia minima e una logica interna esplicita, cosi da facilitare sia il riuso sia la verifica attraverso test.

7.1 LED

Scopo Il file `led.h` fornisce primitive di attuazione per scrivere su pin digitali o PWM. Centralizzare queste funzioni evita duplicazioni nei moduli di controllo.

Flusso `led(pin, state)` esegue una singola `digitalWrite` e ritorna `true`. `ledDimming(pin, pwm)` esegue `analogWrite` con lo stesso schema. La semantica di ritorno uniforme semplifica i test.

Sicurezza Le funzioni non implementano logiche di sicurezza, ma sono utilizzate in un contesto dove i moduli chiamanti validano i propri input. Questo evita che un errore sensore venga trasformato in un attuatore attivo.

Scelte La decisione di mantenere funzioni semplici e senza stato riduce la complessita e migliora la prevedibilita del comportamento in test con mock.

7.2 Servomotore e tornello

Scopo Il modulo `servomotor.h` protegge il servo tramite `antiSufferingServo`. Il modulo `turnstile.h` gestisce ingressi e uscite, mantenendo un contatore di persone.

Flusso In `Turnstile::begin` vengono configurati i pin dei pulsanti e si aggancia il servo. In `update()`, si leggono i pulsanti IN/OUT, si applicano i vincoli di capienza e si muove il servo in apertura e chiusura.

Sicurezza `antiSufferingServo` impedisce di superare il range 0–180, logga un messaggio su Serial e ritorna `false`. Questo riduce il rischio di danni meccanici. Il contatore impedisce ingressi oltre il massimo o uscite sotto zero.

Scelte L'uso di un contatore interno evita dipendenze esterne e rende il modulo autonomo. Il controllo dei limiti prima della movimentazione del servo e una scelta di sicurezza per evitare attuazioni non necessarie.

7.3 Semaforo

Scopo `stoplight.h` traduce lo stato di capienza in segnali visivi immediati.

Flusso `begin()` configura i pin come output. `update(currentPeople, maxPeople)` decide se accendere il verde o il rosso in base al confronto.

Sicurezza Non sono previste modalita intermedie: in caso di condizioni non previste, il modulo mantiene sempre una delle due uscite attive, evitando ambiguita per l'operatore.

Scelte La logica binaria rende il comportamento facile da verificare e minimizza gli errori di interpretazione.

7.4 Temperatura e termostato

Scopo `temperature.h` converte la lettura analogica dell'NTC in gradi Celsius. `thermostat.h` decide la modalita di clima in base a una soglia centrale con tolleranza.

Flusso Il lettore di temperatura verifica i valori estremi e ritorna `-999.0` se rileva un errore. Il termostato legge la temperatura, determina l'azione richiesta e attiva il riscaldamento o il raffreddamento.

Sicurezza Se la lettura e invalida, il termostato spegne entrambe le uscite e ritorna `false`. Inoltre, la pausa temporale tra cambi di stato previene il toggling rapido che potrebbe usurare i componenti.

Scelte La tolleranza di 1.0 C crea una finestra di neutralita che stabilizza il sistema. La pausa e un compromesso tra reattivita e stabilita, adatta a un ambiente reale.

7.5 Umidita e controllo

Scopo `humidity.h` incapsula l'uso del DHT22 e fornisce una lettura di umidita. `humidifier.h` gestisce l'attuatore in base al target.

Flusso Il sensore viene inizializzato con `beginHumiditySensor`. `readHumidity()` ritorna il valore attuale o `-999.0` in caso di errore. Il controllo umidita confronta la lettura con il target e applica una tolleranza.

Sicurezza In caso di errore sensore, l'uscita viene spenta. La tolleranza evita oscillazioni continue intorno al setpoint.

Scelte L'uso di un valore sentinella unico per gli errori rende semplice l'integrazione con il display e con la diagnostica.

7.6 Illuminazione

Scopo `photoresistor.h` stima il lux da una fotoresistenza. `lighting_control.h` regola la plafoniera tramite PWM.

Flusso La lettura analogica viene trasformata in lux con un modello semplificato. Il controllo calcola la differenza dal target e decide se attivare il PWM.

Sicurezza Le letture fuori range sono gestite con valori minimi o massimi, evitando divisioni per zero. Il PWM e sempre limitato nel range 0–255.

Scelte In simulazione Wokwi il PWM e impostato a 255 quando serve luce, per ottenere un comportamento visivo chiaro e prevedibile durante i test.

Capitolo 8— Display LCD

Il display 16x2 usa un layer di astrazione definito in `display.h` (classe `DisplayInterface`) e la logica applicativa in `lcd.h` (classe `DisplayPanel`). Questa separazione permette di testare la logica senza dipendere direttamente dall'hardware.

8.1 Pagine

Il pannello ruota tre pagine ogni 5 secondi:

- **Data/Ora**: calcolata a partire da `__DATE__`, `__TIME__` e `millis()`.
- **Stato**: temperatura, umidita e conteggio persone.
- **Diagnostica**: messaggi di errore o anomalie.

La pagina errori mostra in modo prioritario le condizioni critiche, facilitando l'intervento rapido.

8.2 Calcolo data e ora

Non essendo presente un RTC, l'orario e calcolato combinando il timestamp di compilazione con l'incremento di `millis()`. La base e determinata al costruttore, cosi da offrire una data iniziale coerente e un avanzamento temporale continuo.

8.3 Aggiornamento

Il refresh e limitato a 1 secondo per evitare flicker. Ogni update legge i valori dai moduli `Thermostat`, `HumidifierControl` e `LightingControl`. La pagina cambia ogni 5 secondi per lasciare il tempo di leggere ciascuna informazione.

8.4 Sicurezza e robustezza

Se un valore sensore e invalido, il display mostra `ERR` o un messaggio esplicito. Questo evita interpretazioni errate e rende evidenti i guasti.

Capitolo 9— Implementazione passo-passo

Questo capitolo descrive il flusso operativo per ogni funzionalita, con frammenti di codice e spiegazioni dettagliate.

9.1 Inizializzazione globale

Nel file principale i moduli sono istanziati con parametri chiave e con i pin definiti in modo centralizzato. Questa scelta rende immediata la lettura della configurazione.

```
const int PIN_SERVOMOTOR = 3;
const int PIN_GREEN_LIGHT = 4;
const int PIN_RED_LIGHT = 5;

Turnstile entranceTurnstile(PIN_IN_BUTTON, PIN_OUT_BUTTON, 5);
Stoplight trafficLight(PIN_GREEN_LIGHT, PIN_RED_LIGHT);
Thermostat mainThermostat(PIN_TEMPERATURE_SENSOR, 20.0, PIN_YELLOW_LIGHT,
    PIN_BLU_LIGHT);
LightingControl galleryLighting(PIN_PHOTORESISTOR, PIN_PLAFONIERE, 200);
HumidifierControl galleryHumidifier(IS_HUMIDIFIER_LED, 65.0);
DisplayPanel galleryDisplay(PIN_LCD_RS, PIN_LCD_EN, PIN_LCD_D4, PIN_LCD_D5, PIN_LCD_D6,
     PIN_LCD_D7, 5);
```

Motivazione I target e la capienza sono centralizzati per ridurre errori e per consentire una taratura rapida.

9.2 Tornello e conteggio persone

Il tornello legge i pulsanti e muove il servo con protezione meccanica.

```
if (inState == HIGH && _currentPeople < _maxPeople) {
    moveServo(90);
    _currentPeople++;
    delay(1000);
    moveServo(-90);
}
```

Passi 1) lettura ingressi, 2) controllo capienza, 3) apertura servo, 4) aggiornamento contatore, 5) chiusura servo.

Sicurezza Il contatore evita ingressi oltre il limite e la funzione `antiSufferingServo` impedisce movimenti fuori range.

9.3 Semaforo di capienza

Il semaforo traduce il conteggio in uno stato immediatamente leggibile.

```
if (currentPeople < maxPeople) {
    led(_greenPin, HIGH);
```

```
    led(_redPin, LOW);
} else {
    led(_greenPin, LOW);
    led(_redPin, HIGH);
}
```

Motivazione Due soli stati riducono l'ambiguita per l'operatore.

9.4 Termostato

Il termostato legge la temperatura e decide l'azione con tolleranza e pausa anti-toggle.

```
if (temperature_celsius < (_desiredTemperature - tolerance)) {
    desired_action = 1;
} else if (temperature_celsius > (_desiredTemperature + tolerance)) {
    desired_action = 2;
} else {
    desired_action = 0;
}
```

Passi 1) lettura sensore, 2) validazione, 3) confronto con target, 4) scelta azione, 5) attuazione con pausa.

Sicurezza In caso di lettura invalida, entrambe le uscite sono spente e il metodo ritorna `false`.

9.5 Controllo umidita

Il modulo confronta la lettura del DHT22 con il target e una tolleranza fissa.

```
if (currentHumidity > (_targetHumidity + tolerance)) {
    led(_humidifierPin, HIGH);
} else if (currentHumidity < (_targetHumidity - tolerance)) {
    led(_humidifierPin, LOW);
} else {
    led(_humidifierPin, LOW);
}
```

Sicurezza Se il sensore ritorna `NAN`, la lettura diventa `-999.0` e il modulo spegne l'uscita.

9.6 Illuminazione

Il controllo luci usa un modello semplice per tradurre la lettura analogica in lux e decide il PWM.

```
float missingLight = _targetLux - currentLux;
if (missingLight > 0) {
    pwm_value = 255;
} else {
    pwm_value = 0;
}
ledDimming(_dimmerPin, pwm_value);
```

Motivazione In simulazione Wokwi si usa un PWM pieno quando serve luce, per rendere evidente la risposta.

9.7 Display LCD

Il display ruota pagine con informazioni diverse e mostra gli errori con priorita.

```
if (now - _lastPageChange >= 5000) {
    _page = (_page + 1) % 3;
    _lastPageChange = now;
}
```

Passi 1) limitazione refresh, 2) rotazione pagina, 3) lettura valori dai moduli, 4) stampa pagina selezionata.

Sicurezza Le letture invalide sono mostrate come `ERR` o come messaggi di errore, evitando interpretazioni errate.

Capitolo 10— Requisiti e tracciabilita

Questa sezione elenca i requisiti funzionali principali e la loro tracciabilita verso i test unitari. L'obiettivo e dimostrare che le scelte progettuali sono verificate in modo sistematico.

10.1 Requisiti funzionali

- **R1** Il sistema deve contare ingressi e uscite con limite massimo configurabile.
- **R2** Il sistema deve segnalare lo stato di capienza con semaforo verde/rosso.
- **R3** Il sistema deve controllare temperatura con modalita riscaldamento e raffreddamento.
- **R4** Il sistema deve controllare l'umidita con soglia e tolleranza.
- **R5** Il sistema deve controllare l'illuminazione con target in lux.
- **R6** Il display deve mostrare data/ora, stato e diagnostica.
- **R7** In caso di errore sensori, gli attuatori devono essere disattivati.

10.2 Tracciabilita test

Req.	Descrizione	Test correlati
R1	Conteggio persone con limiti	`TurnstileIncrementsAndDecrementsPeople` `TurnstileRespectsMaxPeople`
R2	Semaforo capienza	`StoplightBelowMaxSetsGreen` `StoplightAtMaxSetsRed`
R3	Clima con heating/cooling	`ThermostatHeatingBranch` `ThermostatCoolingBranch`
R4	Controllo umidita	`HumidifierHighHumidityTurnsOnPin` `HumidifierLowHumidityTurnsOffPin`
R5	Controllo illuminazione	`LightingControlWritesPwm`
R6	Pagine display	`DisplayPanelRotatesPages` `DisplayDatePageHasValidWeekdayPrefix`
R7	Fail-safe su errori sensori	`ThermostatSensorErrorTurnsEverythingOff` `HumidifierSensorErrorReturnsFalse`

10.3 Note

I test sono eseguiti con mock Arduino, quindi le verifiche si concentrano sui side effect (pin e buffer LCD) e non sull'hardware reale. Questo approccio garantisce rapidita e ripetibilita.

Capitolo 11— Test

La suite di test usa GoogleTest ed e configurata nella cartella `test/`. L'obiettivo e garantire che ogni modulo reagisca correttamente sia ai casi nominali sia agli edge case, con particolare attenzione alle condizioni di sicurezza.

11.1 Struttura

Il file `CMakeLists.txt` in `test/` scarica GoogleTest via FetchContent. L'eseguibile generato si chiama `pinacoteca_tests` e i mock delle API Arduino sono in `test/mocks/`. L'entry point dei test e `all_tests.cpp`, dove ogni gruppo di funzionalita ha test dedicati.

11.2 Esecuzione

Sono disponibili due modalita:

- Uso diretto di CMake:

```
cd test
cmake -S . -B build
cmake --build build -j
ctest --test-dir build --output-on-failure
```

- Script `run_tests.sh` che gestisce build, pulizia e compilazione Arduino.

11.3 Motivazioni per i test

Di seguito la spiegazione dettagliata delle verifiche, con obiettivo, condizioni e aspettative.

11.3.1 LED

- **Obiettivo**: garantire che le primitive di uscita scrivano sui pin corretti.
- **Condizioni**: invocazioni dirette di `led` e `ledDimming` con valori noti.
- **Aspettative**: `digitalWrite` e `analogWrite` ricevono il valore richiesto.
- **Test**: `LedWriteDigitalState`
  `LedDimmingWritesPwm`

11.3.2 Stoplight

- **Obiettivo**: validare la semantica verde/rosso in base alla capienza.
- **Condizioni**: conteggi sotto e pari al massimo.
- **Aspettative**: verde acceso sotto-capienza, rosso acceso a capienza piena.
- **Test**: `StoplightBelowMaxSetsGreen`
  `StoplightAtMaxSetsRed`

11.3.3 Servomotore

- **Obiettivo**: impedire movimenti oltre i limiti meccanici.
- **Condizioni**: comandi con angolo valido e con angolo oltre 0–180.
- **Aspettative**: movimento consentito nel range, blocco e log Serial fuori range.
- **Test**: `AntiSufferingServoAllowsSafeMove`
  `AntiSufferingServoBlocksOutOfRange`

11.3.4 Turnstile

- **Obiettivo**: contare correttamente ingressi/uscite e rispettare il massimo.
- **Condizioni**: stati dei pulsanti IN/OUT simulati; massimo persone variabile.
- **Aspettative**: incremento e decremento coerenti, nessun superamento del limite.
- **Test**: `TurnstileIncrementsAndDecrementsPeople`
  `TurnstileRespectsMaxPeople`
  `TurnstileBeginConfiguresPins`

11.3.5 Temperature

- **Obiettivo**: gestire letture anomale del sensore NTC.
- **Condizioni**: valori analogici ai limiti (0 e 1023) e un valore intermedio.
- **Aspettative**: ritorno del sentinella `-999.0` ai limiti, valore finito in condizioni normali.
- **Test**: `TemperatureReaderReturnsErrorAtExtremes`
  `TemperatureReaderReturnsFiniteForValidInput`

11.3.6 Thermostat

- **Obiettivo**: verificare logica heating/cooling e comportamento fail-safe.
- **Condizioni**: lettura valida sopra/sotto target, e lettura invalida.
- **Aspettative**: attivazione del pin corretto, spegnimento su errore sensore.
- **Test**: `ThermostatHeatingBranch`
  `ThermostatCoolingBranch`
  `ThermostatSensorErrorTurnsEverythingOff`
  `ThermostatBeginSetsPinModesAndSetGetTarget`

11.3.7 Humidity e HumidifierControl

- **Obiettivo**: validare la lettura del DHT22 e la logica di soglia.
- **Condizioni**: umidita alta, bassa e lettura non valida (`NAN`).
- **Aspettative**: attuatore coerente con la soglia e spegnimento su errore.

- **Test**: `HumidityReadAndError`
  `HumidifierHighHumidityTurnsOnPin`
  `HumidifierLowHumidityTurnsOffPin`
  `HumidifierSensorErrorReturnsFalse`
  `HumidifierSetGetTargetWorks`

11.3.8 Photoresistor e LightingControl

- **Obiettivo**: garantire conversione e PWM coerenti con il target.
- **Condizioni**: valori analogici ai limiti e valori intermedi.
- **Aspettative**: lux clampati ai bordi e PWM coerente (255 con poca luce, 0 con luce alta).
- **Test**: `PhotoresistorLuxEdgeCases`
  `LightingControlWritesPwm`
  `LightingControlBeginAndSetGetTarget`

11.3.9 DisplayInterface

- **Obiettivo**: verificare il layer LCD astratto.
- **Condizioni**: chiamate a `begin`, `clear`, `setCursor`, `print`.
- **Aspettative**: il buffer del mock LCD contiene le stringhe attese.
- **Test**: `DisplayInterfaceBasicOperations`

11.3.10 DisplayPanel

- **Obiettivo**: verificare rotazione pagine e messaggi diagnostici.
- **Condizioni**: avanzamento del tempo simulato e variazione dei valori sensore.
- **Aspettative**: pagina data/ora, pagina stato e pagina errori mostrate a rotazione.
- **Test**: `DisplayPanelRotatesPages`
  `DisplayDatePageHasValidWeekdayPrefix`
  `DisplayStatusPageShowsTurnstileCountExactly`
  `DisplayErrorPageShowsHumidityHigh`
  `DisplayErrorPageShowsTurnstileError`

11.4 Script run_tests.sh

Opzioni principali:

- `-clean`: elimina la cartella build.
- `-with-arduino`: compila lo sketch con arduino-cli.
- `-upload`: carica lo sketch su una board (richiede `-port`).
- `-board <fqbn>`: seleziona la board, default `arduino:avr:uno`.

Capitolo 12— Istruzioni d'uso

12.1 Build dei test

Per eseguire i test unitari:

```
make test
```

Questo comando invoca lo script e produce l'eseguibile di test nella cartella `test/build`.

Per pulire:

```
make test-clean
```

La pulizia e consigliata quando si cambiano toolchain o si aggiornano le dipendenze.

12.2 Compilazione Arduino

Per compilare lo sketch tramite lo script:

```
./test/run_tests.sh --with-arduino --board arduino:avr:uno
```

Questa modalita esegue prima i test unitari e poi compila lo sketch, riducendo il rischio di caricare un firmware non verificato.

Per caricare sulla board reale:

```
./test/run_tests.sh --upload --board arduino:avr:uno --port /dev/ttyACM0
```

Prima dell'upload, verificare che la porta seriale sia corretta.

12.3 Simulazione Wokwi

Il cablaggio di simulazione e definito in `diagram.json`. Aprire il progetto in Wokwi e avviare la simulazione per vedere il comportamento complessivo. La simulazione e utile per validare il flusso senza hardware fisico.

Capitolo 13— Rischi e failure modes

Questa sezione identifica i principali rischi tecnici e le contromisure implementate. L'obiettivo e dimostrare consapevolezza dei failure modes e una strategia di mitigazione realistica.

13.1 Rischi sensori

Rischio	Effetto	Mitigazione
NTC scollegato o in corto	Lettura errata o infinita	Valore sentinella `-999.0`, uscite spente
DHT22 non risponde	Umidita non valida	Controllo disattivato, errore mostrato su LCD
Fotoresistenza saturata	Lux estremo	Valori clampati (0 o 9999) per evitare divisioni per zero
Pulsanti rumorosi	Conteggio errato	Soglie di capienza e logica di incremento/decremento

13.2 Rischi attuatori

Rischio	Effetto	Mitigazione
Servo oltre corsa	Stress meccanico	`antiSufferingServo` blocca oltre 0–180
Toggle rapido clima	Usura attuatori	Pausa temporale prima del cambio stato
PWM errato	Illuminazione instabile	PWM sempre in range 0–255

13.3 Rischi di integrazione

- **Dipendenze librerie**: mitigato con API semplici e mock locali per test.
- **Incoerenza stati**: mitigata con sequenza fissa nel `loop()`.
- **Diagnostica insufficiente**: mitigata con pagina errori su LCD e log Serial.

Capitolo 14— Timing e prestazioni

Questa sezione descrive le tempistiche chiave del firmware e le motivazioni dietro alle scelte temporali.

14.1 Frequenze di aggiornamento

- **Display**: refresh ogni 1 secondo, cambio pagina ogni 5 secondi.
- **Termostato**: verifica continua con pausa anti-toggle di 5 secondi.
- **Turnstile**: risposta immediata ai pulsanti, con delay di 1 secondo per simulare il passaggio.

14.2 Motivazioni

- Il refresh LCD a 1 secondo evita flicker e riduce scritture ridondanti.
- La pausa del termostato stabilizza il comportamento vicino alla soglia.
- Il delay sul tornello evita doppi conteggi ravvicinati.

14.3 Impatto sul loop

La sequenza dei moduli e breve e deterministica. L'unico delay intenzionale e quello del tornello, scelto per riprodurre l'apertura fisica. In un'evoluzione futura, questo delay puo essere sostituito da una gestione non bloccante basata su `millis()`.

Capitolo 15— Limiti noti e roadmap

15.1 Limiti attuali

- Il controllo PWM della luce in simulazione usa un valore fisso per limiti Wokwi.
- Il tornello usa delay bloccante di 1 secondo durante l'apertura.
- L'orario e stimato senza RTC, basato su `__DATE__` e `__TIME__`.
- Non sono implementati filtri anti-rimbalzo hardware/software sui pulsanti.

15.2 Roadmap

- Introduzione di un RTC per data/ora precise.
- Controllo PWM reale con feedback lux (PID o controllo proporzionale).
- Gestione non bloccante del tornello con macchina a stati.
- Debounce software per pulsanti e logica di evento.
- Espansione test con casi di stress e timing.

Capitolo 16— Tracciabilita requisiti-design-test

Questa sezione collega requisiti, componenti software e test unitari, focalizzandosi esclusivamente sugli aspetti di programmazione e simulazione.

16.1 Matrice di tracciabilita

Req.	Componenti software	Test
R1	`Turnstile`, `servomotor.h`	`TurnstileIncrementsAndDecrementsPeople`, `TurnstileRespectsMaxPeople`
R2	`Stoplight`, `led.h`	`StoplightBelowMaxSetsGreen`, `StoplightAtMaxSetsRed`
R3	`Thermostat`, `temperature.h`	`ThermostatHeatingBranch`, `ThermostatCoolingBranch`
R4	`HumidifierControl`, `humidity.h`	`HumidityReadAndError`, `HumidifierSensorErrorReturnsFalse`
R5	`LightingControl`, `photoresistor.h`	`PhotoresistorLuxEdgeCases`, `LightingControlWritesPwm`
R6	`DisplayPanel`, `DisplayInterface`	`DisplayPanelRotatesPages`, `DisplayInterfaceBasicOperations`
R7	Tutti i moduli di controllo	`ThermostatSensorErrorTurnsEverythingOff`, `HumidifierSensorErrorReturnsFalse`

16.2 Note

La tracciabilita evidenzia che ogni requisito software e coperto da almeno un test e che i moduli sono indipendenti e verificabili in isolamento tramite mock.

Capitolo 17— Validazione software e simulazione

Questa sezione descrive come validare il comportamento del firmware in simulazione e su ambiente di test, senza entrare in aspetti elettrici.

17.1 Strategia di validazione

- **Unit test**: coprono le funzioni di base e la logica dei moduli.
- **Simulazione Wokwi**: verifica end-to-end del ciclo `loop()` con sensori e attuatori virtuali.
- **Test di regressione**: esecuzione ripetuta della suite dopo modifiche al codice.

17.2 Procedure di validazione in simulazione

1. Avviare la simulazione e verificare l'inizializzazione del display.
2. Premere i pulsanti IN/OUT e verificare conteggio persone e semaforo.
3. Forzare temperature alte/basse e osservare LED di heating/cooling.
4. Modificare umidita e osservare attivazione del controllo.
5. Oscurare o illuminare la fotoresistenza e verificare la plafoniera.
6. Controllare la rotazione delle pagine LCD e i messaggi di errore.

17.3 Criteri di accettazione software

- Nessun crash o blocco nel ciclo `loop()`.
- Tutti i moduli rispondono coerentemente ai valori di input simulati.
- Le condizioni di errore disattivano le uscite e mostrano diagnostica.
- La suite di test e eseguita con esito positivo.

Capitolo 18— Calibrazione e limiti di simulazione

Questa sezione descrive aspetti di calibrazione software e limiti della simulazione.

18.1 Calibrazione software

- **Temperatura**: il calcolo NTC usa parametri standard (BETA, RL10). La precisione reale dipende dal sensore e dal cablaggio, ma in simulazione il modello e stabile.
- **Umidita**: il DHT22 in simulazione restituisce valori ideali; in ambiente reale e consigliata una calibrazione empirica.
- **Lux**: la conversione da fotoresistenza e approssimata; il target in lux e un riferimento software piu che una misura certificata.

18.2 Limiti della simulazione

- Il PWM e semplificato (valore pieno quando serve luce).
- Il tempo di risposta dei sensori e idealizzato.
- Alcuni fenomeni reali (rumore, deriva, latenza) non sono riprodotti.

Capitolo 19— Trade-off e scelte progettuali

Questa sezione riassume alcune alternative valutate e le motivazioni della soluzione adottata, limitandosi agli aspetti software.

19.1 Gestione del tempo

- **Scelta**: uso di `delay` nel tornello e `millis()` per display/termostato.
- **Motivazione**: semplice da comprendere e sufficiente in simulazione.
- **Trade-off**: il `delay` e bloccante; una FSM sarebbe piu scalabile.

19.2 Valori sentinella per errori

- **Scelta**: `-999.0` per errori sensori.
- **Motivazione**: facile da riconoscere e propagare.
- **Trade-off**: richiede controllo esplicito in ogni modulo.

19.3 Architettura modulare

- **Scelta**: moduli indipendenti con API piccole.
- **Motivazione**: facilita test e manutenzione.
- **Trade-off**: maggiore numero di file e interfacce da mantenere.

Capitolo 20— Glossario

Termine	Significato
Fail-safe	Comportamento che porta il sistema in stato sicuro in caso di errore.
FSM	Macchina a stati finiti usata per gestire eventi nel tempo.
Lux	Unita di misura dell'illuminamento.
PWM	Modulatione di ampiezza di impulso per regolare potenza.
Sentinella	Valore speciale che indica errore o stato non valido.
Wokwi	Simulatore online per Arduino e componenti elettronici.

Capitolo 21— Appendice: API pubbliche

Questa appendice riassume le principali classi e funzioni esposte dai moduli. Le API sono progettate per essere minimali e facili da testare, con un pattern comune `begin()` + `update()` per i moduli di controllo.

21.1 Turnstile

Gestisce il conteggio persone e l'attuazione del servo in base agli ingressi dei pulsanti.

- `Turnstile(int inButton, int outButton, int maxPeople)`
- `void begin(Servo* servo)`
- `void update()`
- `int getPeopleCount() const`

21.2 Stoplight

Traduce la capienza in un segnale visivo rosso/verde.

- `Stoplight(int greenPin, int redPin)`
- `void begin()`
- `void update(int currentPeople, int maxPeople)`

21.3 Thermostat

Controlla riscaldamento e raffreddamento con tolleranza e pausa anti-toggle.

- `Thermostat(int sensorPin, float desiredTemperature, int heatingPin, int coolingPin)`
- `void begin()`
- `bool update()`
- `void setTargetTemperature(float newTarget)`
- `float getTargetTemperature() const`
- `float getCurrentTemperature() const`

21.4 HumidifierControl

Controlla l'umidita con gestione del valore sentinella in caso di errore.

- `HumidifierControl(int humidifierPin, float targetHumidity)`
- `void begin()`
- `bool update()`
- `void setTargetHumidity(float newHumidityTarget)`
- `float getTargetHumidity() const`
- `float getCurrentHumidity() const`

21.5 LightingControl

Regola la luce in base al target di lux tramite PWM.

- `LightingControl(int sensorPin, int dimmerPin, int targetLux)`
- `void begin()`
- `bool update()`
- `void setTargetLux(int newLuxTarget)`
- `int getTargetLux() const`
- `float getCurrentLux() const`

21.6 DisplayPanel

Gestisce la UI LCD con pagine a rotazione e messaggi di errore.

- `DisplayPanel(int rs, int en, int d4, int d5, int d6, int d7, int maxPeople)`
- `void begin()`
- `void update(int currentPeople, Thermostat&, HumidifierControl&, LightingControl&)`

21.7 Funzioni di utilita

Primitive riusabili per I/O e letture sensori.

- `bool led(int pin, bool state)`
- `bool ledDimming(int pin, int pwm)`
- `float readTemperatureCelsius(int sensorPin)`
- `float readHumidity()`
- `float readLux(int light_sensor_pin)`
- `bool antiSufferingServo(int angle, Servo& s)`

Capitolo 22— Software e dipendenze

22.1 Strumenti

- Arduino IDE o arduino-cli per compilazione e upload.
- CMake >= 3.16 per i test.
- Compilatore C++17 per i test su PC.

L'uso di arduino-cli consente automazione e integrazione con pipeline di test.

22.2 Librerie

- Servo (Arduino)
- LiquidCrystal (Arduino)
- DHT (per DHT22)
- GoogleTest (solo per test)

Le librerie Arduino sono usate tramite header standard, mentre GoogleTest e scaricato dinamicamente nella fase di configurazione CMake.

22.3 Repository

Il progetto e organizzato in:

- `Pinacoteca.ino` per il firmware;
- `lib/` per i moduli;
- `test/` per i test unitari;
- `diagram.json` per la simulazione Wokwi.

Capitolo 23— Controllo remoto, comunicazione App e miglioramenti di stabilita

23.1 Obiettivo del capitolo

Questo capitolo documenta tutte le aggiunte introdotte per la comunicazione remota con app mobile: architettura software, protocollo comandi, strategia di test, ottimizzazioni per i limiti hardware e problemi riscontrati in Wokwi con relative soluzioni.

23.2 Architettura della comunicazione remota

La comunicazione e stata progettata a strati:

- **Astrazione trasporto**: `bluetooth_connection.h` definisce l'interfaccia comune.
- **Trasporto BLE concreto**: `ble_link.h` implementa l'interfaccia per Bluetooth Low Energy.
- **Logica protocollo**: `command_processor.h` interpreta i comandi, valida i payload e aggiorna lo stato.
- **Orchestrazione**: `remote_gateway.h` gestisce seriale, polling BLE e pubblicazione periodica dello stato.

Questa struttura semplifica manutenzione e sostituzione del componente di comunicazione.

23.3 Protocollo utilizzato dall'app

Il protocollo e testuale e pensato per integrazione semplice con App Inventor.

23.3.1 Comandi base

- `PING` → `PONG`
- `VER` → `VER:1`
- `GET:STATE`
- `GET:MODE`
- `MANUAL:ON` / `MANUAL:OFF`

23.3.2 Comandi setpoint

- `SET:TEMP:<valore>` con range $[15.0, 30.0]$
- `SET:HUM:<valore>` con range $[40.0, 80.0]$
- `SET:LUX:<valore>` con range $[50, 1200]$
- `SET:PEOPLE:<valore>` con range $[0, \text{maxPeople}]$

23.3.3 Comandi override manuale

In modalita `MANUAL` l'app puo bypassare la logica automatica:

- Servo: `SERVO:OPEN`, `SERVO:CLOSE`, `SERVO:ANGLE:<0..180>`
- Attuatori: `ACT:GREEN:*`, `ACT:RED:*`, `ACT:HEAT:*`, `ACT:COOL:*`, `ACT:HUMIDIFIER:*`
- PWM luci: `ACT:PLAFONIERE:PWM:<0..255>`

23.3.4 Formato payload stato

Esempio:

```
STATE:T=21.8;H=58.0;L=170;P=3;S=90;M=MANUAL;TT=22.5;TH=60.0;TL=180
```

23.3.5 Gestione errori

Per robustezza lato app sono previsti errori espliciti:

- `ERR:EMPTY`, `ERR:UNKNOWN`, `ERR:MODE`, `ERR:SERVO`
- Errori di range: `ERR:RANGE:*`
- Errori di formato: `ERR:FORMAT:*`
- Protezione overflow seriale: `ERR:TOO_LONG`

23.4 Strategia di validazione e test

La suite test e stata estesa con casi dedicati al controllo remoto:

- correttezza protocollo (comandi validi e invalidi),
- valori limite per tutti i range,
- payload numerici malformati,
- transizioni AUTO/MANUAL,
- input seriale frammentato,
- input seriale troppo lungo,
- trasporto disconnesso,
- coda comandi su cicli multipli.

Nei test viene usato un trasporto finto per validare la logica anche senza hardware BLE reale.

23.5 Ottimizzazioni software per stabilita

Sono state introdotte diverse ottimizzazioni per ridurre rischi di instabilita:

- riduzione footprint RAM con tipi interi piu piccoli per pin e contatori,
- rimozione di comportamento bloccante nel tornello e chiusura servo non bloccante,
- riduzione churn dinamico delle stringhe nella generazione stato,
- parsing numerico rigoroso per evitare conversioni non sicure,

- separazione output build per target diversi,
- workflow dedicati per simulazione e hardware reale.

Queste modifiche migliorano prevedibilita runtime e riducono problemi in simulazione/deploy.

23.6 Problemi riscontrati in Wokwi e soluzioni

23.6.1 Problema A: mismatch board/firmware

Se la board simulata e il firmware compilato non coincidono, la simulazione risulta non funzionante.

Soluzione: output separati:

- Wokwi in `build-wokwi/`
- hardware UNO R4 in `build-r4/`

23.6.2 Problema B: chiave duplicata in `wokwi.toml`

Una chiave `elf` duplicata causava errore di parsing TOML.

Soluzione: mantenere una sola entry `elf` e validare il file.

23.6.3 Problema C: servo non visibile in movimento

Dopo la rimozione dei delay bloccanti, il servo poteva aprire/chiudere troppo rapidamente nello stesso ciclo.

Soluzione: introduzione chiusura temporizzata non bloccante e trigger su fronte pulsante.

23.7 Workflow build completo (comando unico)

Il progetto supporta un comando unico che esegue test completi e doppia compilazione:

```
make full-test
```

Questo comando esegue:

1. test completi,
2. compilazione firmware Wokwi,
3. compilazione firmware UNO R4 WiFi.

23.8 Test app senza hardware

E stato aggiunto un mock server locale per iniziare test App Inventor senza scheda fisica:

- script: `tools/mock_arduino_server.py`
- comando: `make mock-server`

Con questo approccio puoi validare UI, parsing e flussi comandi prima dei test BLE reali.

23.9 Nota finale di deploy

Prima del rilascio finale e comunque necessario validare BLE su hardware UNO R4 WiFi reale, per verificare radio, latenza e comportamento in condizioni reali.